

Stock Management Module — Implementation Guide

Greenfield feature guide for a new `inventory` backend domain implementing stock (inventory) management for Finguard 3.0. Written to match the codebase's existing conventions — the event-sourced ledger pattern from `finance/invoice_events`, the per-domain layering (`models` → `repository` → `service` → `router`), RBAC via `identity/permissions`, audit instrumentation, the sequential Alembic chain, and the OpenAPI sync-types gate.

Naming: the domain package is `inventory` (avoids collision with the finance stock-of-cash mental model and reads well as `Inventory` in the UI/API). It is the "Stock Management" module the product calls for; use "Stock" in user-facing copy, `inventory` in code/routes (`/api/v1/inventory`).

1. Scope & goals

In scope (v1):

- A **product catalog** (SKU, name, unit of measure, category, reorder policy, cost/selling price).
- An **append-only stock movement ledger** (receipts, issues, adjustments, transfers, sales/returns) as the single source of truth.
- A **materialized stock level** per product (on-hand, reserved, average cost), derived by folding movements — mirrors how `invoices` projects `invoice_events`.
- **Weighted-average cost valuation** and an inventory valuation report.
- **Reorder-level alerts** (low stock / stockout) via the existing `alerts` domain.
- Integration seams to **finance** (a purchase receipt ↔ an `Expense`; a sale ↔ invoice line COGS) and **intelligence** (watchdog/forecast can read stock).

Explicitly deferred (backlog — note but don't build in v1):

- Multi-location / multi-warehouse transfers (design the schema for it, run v1 single default location).
- FIFO / batch-lot / serial-number valuation and expiry tracking.
- Purchase-order and goods-received-note (GRN) workflow.
- Barcode scanning UI.

Non-negotiable invariants:

1. On-hand quantity is **never** stored as a free-standing mutable number that code increments ad-hoc — it is a projection of the movement ledger, exactly like invoice balances. Every quantity change is an immutable movement row.
2. Stock can **never go negative** for a physical `ISSUE`/`SALE` (guarded in the service under a row lock); an operator can force it only via an explicit `ADJUSTMENT` with a reason.

3. Money uses `Decimal + Numeric(18, 2)`; quantities use `Numeric(18, 3)` (to allow kg/litre fractions). Never floats.
-

2. Where it fits in the codebase

```
backend/src/domains/inventory/          # NEW domain package (mirror
crm/finance)
├── __init__.py
├── types.py                            # enums: MovementType, MovementReason, UnitOfMeasure
├── models.py                          # Product, StockLevel, StockMovement (+ StockLocation,
deferred use)
├── schemas.py                        # Pydantic request/response models
├── repository.py                     # data access (ProductRepository, StockRepository)
├── service.py                        # business logic + the movement/projection transaction
├── events.py                         # (optional) fold helpers if you split projection out
└── router.py                         # FastAPI endpoints under /api/v1/inventory

backend/alembic/versions/0018_inventory.py  # NEW migration (next in the chain)
backend/tests/domains/inventory/           # NEW tests
frontend/src/lib/api/inventory.ts          # NEW typed API client
frontend/src/app/dashboard/inventory/      # NEW UI route(s)
```

Files you must also touch (registration points):

- `backend/src/main.py` — import and `include_router(inventory_router, prefix="/api/v1/inventory", tags=["inventory"])` (follow the existing block).
 - `backend/alembic/env.py` — add `import src.domains.inventory.models` # noqa: F401 so `autogenerate/Base.metadata` sees the tables.
 - `backend/src/domains/identity/permissions.py` — add `INVENTORY_READ / INVENTORY_WRITE` (and optionally `INVENTORY_ADJUST`) to the `Permission` enum and the role frozensets.
 - `backend/src/domains/identity/dependencies.py` — add `RequireInventoryRead / RequireInventoryWrite` annotated deps (copy the `RequireCrm*` pattern).
 - `backend/src/domains/audit/models.py` — add `AuditAction` values (`STOCK_RECEIVED, STOCK_ISSUED, STOCK_ADJUSTED, PRODUCT_CREATED, ...`).
 - `frontend/src/lib/api/endpoints.ts` — add an `INVENTORY` endpoint group.
 - Regenerate `frontend/src/lib/api/generated/schema.d.ts` (OpenAPI gate — see §9).
-

3. Data model

3.1 Enums (inventory/types.py)

```
import enum

class UnitOfMeasure(enum.StrEnum):
    EACH = "each"
    KG = "kg"
    LITRE = "litre"
    METRE = "metre"
    BOX = "box"
    PACK = "pack"

class MovementType(enum.StrEnum):
    """The signed direction is derived from the type, not stored free-form."""
    RECEIPT = "receipt"          # +qty  purchase / restock (often ⇔ an Expense)
    ISSUE = "issue"              # -qty  internal consumption / write-off
    SALE = "sale"                # -qty  sold to a customer (often ⇔ an Invoice
    line)
    RETURN_IN = "return_in"      # +qty  customer return to stock
    ADJUSTMENT = "adjustment"    # ±qty  stock-take correction (requires reason)
    TRANSFER = "transfer"        # ±qty  between locations (deferred: single-loc
    v1)

# Types that ADD to on-hand; everything else subtracts.
INBOUND = frozenset({MovementType.RECEIPT, MovementType.RETURN_IN})

class MovementReason(enum.StrEnum):
    PURCHASE = "purchase"
    SALE = "sale"
    DAMAGE = "damage"
    THEFT = "theft"
    STOCK_TAKE = "stock_take"
    EXPIRY = "expiry"
    CORRECTION = "correction"
    OTHER = "other"
```

3.2 Tables (inventory/models.py)

Follow the exact column idioms already in `finance/models.py`: UUID PK with `default=uuid.uuid4`, `Numeric` for money/qty, `Enum(...)` for `StrEnums`, `server_default=func.now()` timestamps, `onupdate` for `updated_at`, `CheckConstraint/UniqueConstraint` for invariants.

Product — the catalog entry.

column	type	notes
id	UUID PK	
sku	String(64) unique, indexed	business key
name	String(255) not null	
description	Text nullable	
unit	Enum(UnitOfMeasure)	default EACH
category	String(100) nullable,	

column	type	notes
	indexed	
cost_price	Numeric(18,2)	last/standard purchase price (informational)
selling_price	Numeric(18,2)	
reorder_level	Numeric(18,3) default 0	low-stock threshold
reorder_quantity	Numeric(18,3) default 0	suggested restock qty
barcode	String(64) nullable, indexed	future scanner
is_active	Boolean default True	soft-delete / discontinue
created_at / updated_at	tz DateTime	server_default / onupdate

StockLevel — the **projection** (materialized on-hand), one row per product (per location once multi-location lands). This is the row writers lock.

column	type	notes
id	UUID PK	
product_id	UUID FK → products.id, unique(+location)	
location_id	UUID FK nullable	v1 = a single default location or NULL
quantity_on_hand	Numeric(18,3) not null default 0	fold of movements
quantity_reserved	Numeric(18,3) not null default 0	allocated to open orders
average_cost	Numeric(18,2) not null default 0	weighted-avg valuation
updated_at	tz DateTime onupdate	

Constraints:

```
CheckConstraint("quantity_on_hand >= 0", name="ck_stock_levels_nonneg_on_hand")
CheckConstraint("quantity_reserved >= 0",
name="ck_stock_levels_nonneg_reserved")
UniqueConstraint("product_id", "location_id",
name="uq_stock_levels_product_location")
```

StockMovement — the **append-only ledger** (source of truth). Never updated or deleted. Model it on `InvoiceEvent`: a per-product monotonic sequence with (product_id, sequence) unique gives a gap-free, replayable history; writers serialise on the `StockLevel` row's `FOR UPDATE` lock.

column	type	notes
id	UUID PK	
product_id	UUID FK → products.id, indexed	
location_id	UUID FK nullable	
sequence	Integer not null	per-product version, starts at 1
movement_type	Enum(MovementType) not null	
reason	Enum(MovementReason) nullable	required for ADJUSTMENT
quantity	Numeric(18, 3) not null	always positive ; direction from type
unit_cost	Numeric(18, 2) nullable	required for RECEIPT (drives avg cost)
balance_after	Numeric(18, 3) not null	on-hand snapshot after applying (audit/debug)
reference_type	String(50) nullable	e.g. expense, invoice, receipt_scan
reference_id	UUID nullable, indexed	link to the finance object
note	Text nullable	
created_by	UUID nullable	acting user (audit actor)
created_at	tz DateTime server_default	

```
UniqueConstraint("product_id", "sequence",
name="uq_stock_movements_product_seq")
CheckConstraint("quantity > 0", name="ck_stock_movements_positive_qty")
```

Why a ledger + projection (not just a mutable count): it's the pattern the codebase already trusts for money (invoice_events → invoices). It gives a full audit trail ("why is on-hand 7?"), makes stock-take reconciliation and valuation reports derivable, and makes concurrency correctness a single well-understood lock. Reuse the mental model reviewers already know.

4. Service layer — the one transaction that matters

inventory/service.py holds the business logic and owns the commit (same shape as CRMService/FinanceService). The critical method is apply_movement, which must be **atomic and race-safe**, mirroring FinanceService.apply_reconciled_payment's lock-then-project approach.

```
async def apply_movement(self, cmd: StockMovementCreate, *, actor_id) ->
StockMovement:
    async with self._session.begin():                                # one transaction
```

```

UPDATE level = await self._repo.get_level_for_update(      # SELECT ... FOR
    cmd.product_id, cmd.location_id
)
absent      # create at 0 if
signed = cmd.quantity if cmd.movement_type in INBOUND else -cmd.quantity
new_on_hand = level.quantity_on_hand + signed

# Invariant #2: physical outflow can't oversell.
if new_on_hand < 0 and cmd.movement_type is not MovementType.ADJUSTMENT:
    raise UnprocessableError(
        f"Insufficient stock: on-hand {level.quantity_on_hand}, "
        f"requested {cmd.quantity}"
    )
if cmd.movement_type is MovementType.ADJUSTMENT and cmd.reason is None:
    raise UnprocessableError("Adjustment requires a reason")

# Weighted-average cost update on inbound-with-cost.
if cmd.movement_type in INBOUND and cmd.unit_cost is not None:
    level.average_cost = _weighted_avg(
        level.quantity_on_hand, level.average_cost, cmd.quantity,
cmd.unit_cost
    )
    level.quantity_on_hand = new_on_hand

seq = await self._repo.next_sequence(cmd.product_id)  # max(seq)+1
under lock
    movement = StockMovement(
        **cmd.model_dump(), sequence=seq,
        balance_after=new_on_hand, created_by=actor_id,
    )
    self._repo.add(movement)
# commit released the lock; caller records audit + fires alerts (see below)
return movement

```

Helpers:

- `_weighted_avg(qty_before, avg_before, qty_in, cost_in)` → new average cost. Guard the zero/negative denominator.
- `get_level_for_update` uses `select(StockLevel).where(...).with_for_update();` create the row at zero (get-or-create) inside the same transaction if missing.
- `next_sequence = SELECT COALESCE(MAX(sequence), 0)+1 FROM stock_movements WHERE product_id = :pid` — safe because the `StockLevel FOR UPDATE` lock serialises all writers for that product.

Post-commit side effects (do them like the CRM router does):

- After a successful `ISSUE`/`SALE`/`ADJUSTMENT` that drops on-hand $\leq$ `reorder_level`, create a low-stock **Alert** via the `alerts` domain (best-effort; don't let an alert failure roll back the movement).
- Record an **audit** entry with `AuditService(db).record_user_action_safe(...)` (`STOCK_RECEIVED` / `STOCK_ISSUED` / `STOCK_ADJUSTED`), `resource="product"`, `resource_id=product_id`, `metadata {sku, qty,`

`movement_type, balance_after}`.

Convenience service methods wrapping `apply_movement`:

`receive_stock`, `issue_stock`, `record_sale`, `adjust_stock`, plus catalog
CRUD (`create_product`, `update_product`, `list_products`, `get_product`, and
a soft-delete via `is_active`) copied from `CRMService`'s shape.

5. API surface (`inventory/router.py`)

Use `DBSession = Annotated[AsyncSession, Depends(get_db)]` and the
new `RequireInventory*` deps exactly like `crm/router.py`. Suggested endpoints:

Method	Path	Permission	Purpose
POST	<code>/products</code>	<code>INVENTORY_WRITE</code>	create catalog product
GET	<code>/products</code>	<code>INVENTORY_READ</code>	list (paginate limit/offset, filter category, low_stock=true, q)
GET	<code>/products/{id}</code>	<code>INVENTORY_READ</code>	product + current <code>StockLevel</code>
PATCH	<code>/products/{id}</code>	<code>INVENTORY_WRITE</code>	update catalog fields
POST	<code>/products/{id}/movements</code>	<code>INVENTORY_WRITE</code>	apply a movement (receipt/issue/sale/return)
POST	<code>/products/{id}/adjust</code>	<code>INVENTORY_ADJUST*</code>	stock-take adjustment (reason required)
GET	<code>/products/{id}/movements</code>	<code>INVENTORY_READ</code>	movement history (audit trail)
GET	<code>/levels</code>	<code>INVENTORY_READ</code>	on-hand snapshot across products
GET	<code>/reports/valuation</code>	<code>INVENTORY_READ</code>	$\Sigma(\text{on_hand} \times \text{average_cost})$, by category
GET	<code>/reports/low-stock</code>	<code>INVENTORY_READ</code>	products at/below reorder level

* If you don't want a third permission, gate `/adjust` on `INVENTORY_WRITE` and rely on the
mandatory reason + audit trail. Adjustments are higher-trust (they can create stock from nothing),
so a separate grant held by managers+ is the cleaner separation-of-duties choice — mirror how
`FINANCE_RECONCILE` is split from `FINANCE_WRITE`.

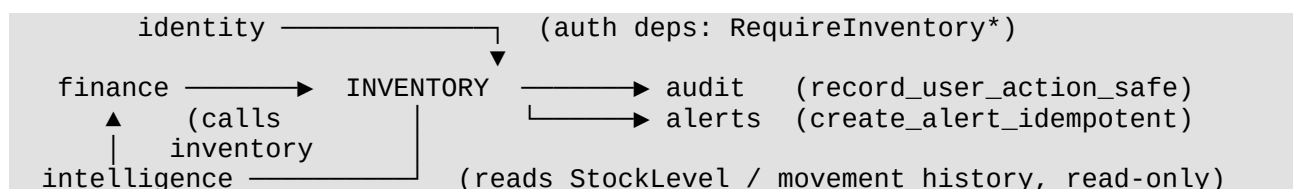
Pydantic schemas (`inventory/schemas.py`): `ProductCreate`,
`ProductUpdate`, `ProductResponse`, `StockLevelResponse`,

StockMovementCreate, StockMovementResponse, ValuationReport, LowStockItem. Add field_validators: quantity > 0, unit_cost required when movement_type == RECEIPT, reason required when movement_type == ADJUSTMENT.

6. Module interactions

6.1 Dependency direction (who imports whom)

The architecture-boundary test forbids arbitrary cross-domain imports. Keep **inventory** a **low-level, mostly-leaf** domain: it depends only on **core + identity** (auth deps) + **audit + alerts** — the same footprint every other business domain has. **Finance and intelligence depend on inventory, not the reverse.** This one-way arrow is what keeps the seams testable and prevents an import cycle.



Rule of thumb: if inventory would need to **import** **finance**, invert it — expose an inventory service method **finance** calls, or pass the **finance** object's **id** in as a **reference_id** (inventory stores the link, never the **finance** model).

6.2 Integration mechanism (today vs later)

- **v1 = synchronous in-process service calls**, inside the caller's DB transaction. This matches **finance** today: the invoice projection is folded synchronously (**finance/events.py** notes the outbox → RabbitMQ consumer is a deliberate *follow-up*, not yet live). So a **finance** flow that also moves stock calls **InventoryService(session).apply_movement(...)** on the **same session** → one atomic commit, no eventual-consistency window.
- **Later = event-driven**, once the outbox/RabbitMQ consumer lands: emit a **stock.received / stock.sold** domain event and let **inventory** subscribe. Don't build this in **v1** — but keep **apply_movement** idempotent-friendly (the (**reference_type**, **reference_id**) pair lets a consumer dedupe replays).

6.3 Interaction matrix

Counterparty	Direction	Trigger	Mechanism	Data exchanged
identity	inventory ←	every request	RequireInventoryRead/Write deps	authenticated User, role
audit	inventory →	after every movement /	AuditService.record_user	actor, AuditAction,

Counterparty	Direction	Trigger	Mechanism	Data exchanged
		product write	<code>_action_safe</code>	sku, qty, balance_after
alerts	inventory →	on-hand crosses reorder_level	<code>AlertService.create_alert_idempotent</code>	product, on-hand, threshold
finance (purchase)	finance → inventory	expense that is a stock purchase	sync service call	product_id, qty, unit_cost, reference=(expense, id)
finance (sale/COGS)	finance → inventory + inventory → finance	invoice sale of a tracked product	sync service call + COGS read	product_id, qty, reference=(invoice, id); back: average_cost
intelligence	intelligence → inventory	agent run (watchdog/forecast)	read-only query	StockLevel, movement history
receipt scanner (Agent A)	UI-mediated	scanned purchase receipt	suggest → operator confirms	proposed product match + unit_cost

6.4 Contracts per counterparty

identity / RBAC (inbound): routers depend on

`RequireInventoryRead / RequireInventoryWrite` (and `INVENTORY_ADJUST` for stock-take). No inventory code touches auth internals — it only receives the resolved User.

audit (outbound, best-effort): mirror `crm/router.py` — build the response model first, then `await AuditService(db).record_user_action_safe(...)`. A failed audit write must never roll back a committed movement.

alerts (outbound, best-effort): use `create_alert_idempotent` (already in `alerts/service.py`) keyed on `(product_id, "low_stock")` so a product sitting below reorder level doesn't spawn a new alert on every subsequent issue. Fire **after** the movement commits.

finance — purchases (finance → inventory):

- Expense today has `category / amount / vault / mpesa_trans_id / invoice_id` and **no product link** — so *no finance schema change is required* for v1. Inventory stores the linkage from its side: when an operator records a stock purchase, create a RECEIPT movement with `reference_type="expense"`, `reference_id=<expense.id>`, `unit_cost=<from expense>`.

- Sequencing: create the Expense and the RECEIPT in the **same session/commit** so a crash can't leave stock received without the expense (or vice versa).
- The receipt-scanner path (POST /intelligence/receipts/scan → ReceiptExpenseCreate) can, after the user confirms, additionally propose a product match (by name/SKU) and a receipt movement — operator-confirmed, never auto-applied.

finance — sales & COGS (bidirectional):

- **⚠ Prerequisite/accuracy note:** InvoiceCreate currently has **no line items** (just subtotal/tax/total). So "decrement stock per invoice line" is *not* a drop-in seam. Two options:
 1. **v1 (recommended, no finance change):** an explicit inventory record_sale endpoint/movement that references the invoice by id (reference_type="invoice"). The UI links the two; stock and receivable stay separate aggregates joined by reference_id.
 2. **Later:** add an InvoiceLine table to finance (product_id, qty, unit_price) and drive SALE movements + COGS off it. This is a finance-side change; schedule it as its own ticket.
- **COGS read (inventory → finance):** finance's reports (finance/reports.py) read $\text{average_cost} \times \text{qty_sold}$ from inventory to compute cost of goods sold / gross margin. Inventory exposes this as a query method; finance calls it.
- **Reservation:** quantity_reserved lets a *draft/sent* invoice hold stock without decrementing on-hand; convert reserved → issued when the sale is confirmed, or release it on invoice cancellation. Optional in v1.

intelligence (read-only): Agent E (watchdog) can flag stockout risk from StockLevel; Agent D (forecaster) can consume movement history for demand forecasting. These are **read-only** queries — no agent mutates stock in v1, and no new agent is added. If/when an agent should *act* on stock, route it through the same apply_movement service (never a direct table write) so the ledger and guards still hold.

6.5 Journey sequence flows

Purchase → stock receipt (single atomic transaction):

```
Operator → POST /finance/expenses (or confirms a scanned receipt)
├── FinanceService.create_expense(...)
├── InventoryService.apply_movement(RECEIPT, qty, unit_cost, reference=(expense, id))
└── audit STOCK_RECEIVED → (no alert; on-hand went up)
```

} same session
} one commit

Sale → stock issue:

```
Operator → POST /inventory/products/{id}/movements (SALE, reference=(invoice,id))
├── apply_movement: lock StockLevel, guard non-negative, project on-hand
├── commit → audit STOCK_ISSUED
└── if on-hand ≤ reorder_level: alerts.create_alert_idempotent(low_stock)
```

→ finance reports later read average_cost for COGS

7. Migration

(backend/alembic/versions/0018_inventory.py)

- `down_revision = "0017_classification_feedback"` (current head — verify with `alembic heads` before writing).
 - Follow the **idempotent** style in 0017 (`inspector.has_table(...)` guards, `CREATE ... IF NOT EXISTS`) so `alembic upgrade head` is safe on a partially migrated DB (the repo requires migrations to run clean on a fresh DB).
 - These are core business tables — put them in the **default (public) schema** like `invoices/customers`, *not* the `finquard` schema (that schema is for the intelligence/tuning side tables).
 - Create `products`, `stock_levels`, `stock_movements` with the indexes and constraints from §3. Add indexes: `products(sku)`, `products(category)`, `stock_movements(product_id, sequence)`, `stock_movements(reference_type, reference_id)`, `stock_levels(product_id)`.
 - Provide a real `downgrade()` that drops the three tables in FK-safe order.
-

8. Frontend

- **Endpoints:** add an `INVENTORY` group to `frontend/src/lib/api/endpoints.ts`.
 - **API client:** `frontend/src/lib/api/inventory.ts` — typed wrappers over `httpClient`, importing generated types (`ApiProduct`, `ApiStockMovement`, ...) from `@/types/api`. Copy the shape of `finance.ts`.
 - **UI route:** `frontend/src/app/dashboard/inventory/` — a product list with on-hand + low-stock badges, a product detail with movement history, and receive/issue/adjust dialogs. Use live TanStack Query hooks (the dashboard's established data-layer pattern — no mock data).
 - Add a nav entry alongside the existing dashboard routes (`invoices`, `payables`, `budgets`, ...).
-

9. OpenAPI sync-types gate (required before CI passes)

New Pydantic response models appear in `components.schemas`, so the generated frontend types must be regenerated or CI's `sync-types:check` fails:

1. Dump the spec from the app

```
python -c "import json; from src.main import app; json.dump(app.openapi(),
open('openapi.json', 'w'))"
# 2. Regenerate (uses node_modules' openapi-typescript — keep the version CI
uses)
cd frontend && npx openapi-typescript ../backend/openapi.json -o
src/lib/api/generated/schema.d.ts
# 3. Commit the regenerated schema.d.ts
```

CSRF double-submit middleware is auto-applied to the new mutating routes — no backend action needed, but the frontend client must send the token (the shared `httpClient` already does).

10. Testing plan (backend/tests/domains/inventory/)

Async DB tests pull the session-scoped `create_tables` fixture automatically (don't add the hermetic no-op override). Model tests on `tests/domains/finance/test_invoices.py`.

Correctness / invariants:

- `receive` then `issue` leaves the expected on-hand and a 2-row ledger with sequence 1,2 and correct `balance_after`.
- **Non-negative guard:** an `ISSUE` exceeding on-hand raises `UnprocessableError` and writes **no** movement (transaction rolled back).
- **Adjustment** can drive on-hand up/down and *requires* a reason.
- **Weighted-average cost:** receive 10@100 then 10@120 → avg 110; a subsequent sale doesn't change avg cost.
- **Concurrency (the money-path test):** two overlapping `issue` calls on the same product that together exceed stock — assert exactly one succeeds and on-hand never goes negative (mirror `test_vault_transfer` overdraw-under-concurrency test in finance).
- **Projection equals fold:** replaying all movements reproduces `quantity_on_hand` (property/oracle test).

API / RBAC:

- A `viewer` gets 403 on any write/adjust; read endpoints return 200.
- Valuation and low-stock reports compute the right totals.
- Audit rows are written for receive/issue/adjust (assert via `AuditService`).

Toolchain: the repo `.venv` is root-owned/broken — run tests via the uv-managed Python 3.12 interpreter (see the backend-venv memo / prior sprints). Run `ruff check + mypy` on the new files; both must be clean.

11. Suggested build order (incremental, each shippable)

Phase	Deliverable	Est
P1	<code>types.py</code> + <code>models.py</code> + migration 0018 + RBAC perms/deps + audit actions + <code>main.py/env.py</code> registration	M
P2	<code>repository.py</code> + <code>service.py</code> (<code>apply_movement</code> , weighted-avg, non-negative guard) + unit/concurrency tests	L
P3	<code>router.py</code> (catalog CRUD + movements + history) + schemas + RBAC/API tests + regenerate <code>schema.d.ts</code>	M
P4	Reports (valuation, low-stock) + reorder-level → <code>alerts</code> integration	M
P5	Frontend <code>inventory.ts</code> + dashboard UI (list, detail, receive/issue/adjust)	L
P6	Finance seams (expense ↔ receipt movement; invoice line ↔ sale/COGS + reservation)	L

If you can only ship one phase: P1+P2 — the ledger, projection, and the race-safe `apply_movement` are the load-bearing core; everything else is CRUD and presentation over that foundation.

12. Acceptance criteria

- On-hand is always the fold of the movement ledger; no code path mutates a quantity outside `apply_movement`.
- Physical outflow can never oversell; the DB `CheckConstraint` is a backstop, the service lock is the primary guard, and a concurrency test proves it.
- Every movement is attributable (actor + audit row) and reversible only by a new compensating movement (ledger is append-only).
- Valuation report = $\Sigma(\text{on_hand} \times \text{average_cost})$; low-stock report lists every product at/below its reorder level.
- `ruff` + `mypy` clean; migrations run on a fresh DB; OpenAPI sync-types gate green; new RBAC permissions enforced (viewer can't mutate).

